



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO
CURSO DE ENGENHARIA INFORMÁTICA

**RELATÓRIO TÉCNICO-CIENTÍFICO DE LÓGICA DE
PROGRAMAÇÃO II**

**SISTEMA DE GESTÃO DE ESTUDANTES EM C
“SIGE-UNIKIVI”**

PERÍODO: MANHÃ

1º ANO

2º SEMESTRE

DOCENTE

MOYO KANIVENGIDIO

UÍGE, 2025/2026

UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO
CURSO DE ENGENHARIA INFORMÁTICA

GRUPO: AJIMN

POR:

1. ANTÓNIO JOAQUIM BERNARDO
2. IGRAÇA NZINGA DE SOUSA KIAWETE
3. JOÃO ISAÍAS TRUVÃO
4. MARIA TUSSONGUA BENEDITH
5. NELSA NZAGE JOÃO

UÍGE, 2025/2026

ÍNDICE

RESUMO	I
LISTA DE FIGURAS	II
0. INTRODUÇÃO.....	1
0.1- OBJECTIVOS	2
0.1.1- Objectivo Geral	2
0.1.2- Objectivos Específicos	2
0.2- METODOLOGIA DE DESENVOLVIMENTO	2
0.3- DIFICULDADES ENCONTRADAS	3
1. CONCEITOS FUNDAMENTAIS UTILIZADOS	3
1.1- ESTRUTURAS DE DADOS (STRUCTS).....	3
1.2- ARRAYS (VETORES) E COMPRESSÃO DE MEMÓRIA	3
1.3- MODULARIZAÇÃO E PONTEIROS (PASSAGEM POR REFERÊNCIA).....	3
1.4- PERSISTÊNCIA EM FORMATOS TXT E BINÁRIO	4
2. MODELAGEM DO SISTEMA	4
2.1- A STRUCT ESTUDANTE	4
2.2- MAPA DE FLUXO DA ARQUITETURA MODULAR	4
3. ANALISE ALGORÍTMICA E IMPLEMENTAÇÃO	5
3.1- ALGORITMOS DE ORDENAÇÃO	5
3.2- ALGORITMOS DE PESQUISA	6
3.3- ALGORITMO DE ELIMINAÇÃO COM CONFIRMAÇÃO	6
4. MANUAL DE UTILIZAÇÃO E RESULTADOS.....	7
4.1- INICIALIZAÇÃO E LEITURA DO FICHEIRO	7
4.2- FLUXO DE SEGURANÇA E ALTERAÇÕES PENDENTES	7
5.8- APRESENTAÇÃO DOS RESULTADOS	7
5.8.1- Tela de Login	7

5.8.2- Tela de Menu principal	8
5.8.3- Gerir Alunos.....	8
5.8.3.1- Tela de Cadastro.....	8
5.8.3.2- Tela de Gravar dados.....	9
5.8.4- Tela de Listagem de alunos.....	9
5.8.5- Tela de Pesquisa por Matrícula	9
5.8.6- Tela de Pesquisa por nome.....	10
5.8.7- Tela de Ordenação.....	10
5.8.8- Tela de Relatórios.....	10
CONSIDERAÇÕES FINAIS	11
REFERÊNCIAS BIBLIOGRÁFICAS	12

RESUMO

O presente relatório descreve o desenvolvimento do Sistema de Gestão de Estudantes da Universidade Kimpa Vita (**SIGE-UNIKIVI**), uma aplicação de software construída na linguagem C padrão (ANSI C) com a finalidade de automatizar e otimizar os processos de manipulação, ordenação, pesquisa e persistência de dados académicos na Universidade Kimpa Vita. O projeto foca-se na transição das abordagens tradicionais de armazenamento temporário para uma infraestrutura estável de memória RAM aliada a ficheiros de texto (.txt) e binários (.bin). Ao longo do documento, são detalhados os aspetos conceituais de engenharia de software utilizados, tais como a divisão modular da aplicação, a passagem de parâmetros utilizando ponteiros para modificação real em memória, e a implementação comparativa de três algoritmos de ordenação e duas metodologias de pesquisa. Os resultados comprovam que a modularidade e a correta aplicação dos algoritmos não só evitam o desperdício de recursos computacionais, mas garantem a integridade operacional dos dados armazenados.

Palavras-Chave: Linguagem C, Estruturas de Dados, Algoritmos de Ordenação, Pesquisa Binária, Persistência de Ficheiros.

LISTA DE FIGURAS

Figura nº 01: Mapa De Fluxo Da Arquitetura Modular	5
Figura nº 02: Tela de Login	7
Figura nº 03: Tela de Menu principal	8
Figura nº 04: Tela de Cadastro	8
Figura nº 05: Tela de Gravar dados	9
Figura nº 06: Tela de Listagem de alunos	9
Figura nº 07: Tela de Pesquisa por matrícula	9
Figura nº 08: Tela de Pesquisa por nome	10
Figura nº 9: Tela de ordenação	10
Figura nº 10: Tela de Relatórios	10

0. INTRODUÇÃO

No contexto universitário contemporâneo, a gestão de dados de estudantes é uma tarefa essencial que exige precisão e rapidez. Sistemas manuais ou mal estruturados resultam frequentemente em perda de dados, redundância e lentidão no processamento de consultas básicas. Com base nessa problemática, Sistema de Gestão de Estudantes da Universidade Kimpa Vita (**SIGE-UNIKIVI**), um sistema de console robusto fundamentado nos conceitos ministrados na disciplina de Lógica de Programação II.

Ao contrário de abordagens puramente teóricas, o sistema foi desenhado para simular o comportamento de um software de nível profissional, tratando situações críticas como a eliminação acidental de registros, buffer corrompido do teclado e alertas ao utilizador no encerramento da aplicação quando existirem alterações não salvas no disco.

0.1- OBJECTIVOS

0.1.1- Objectivo Geral

- ❖ Capacitar o desenvolvimento de uma plataforma funcional em C que gerencie dados académicos complexos através do acoplamento harmónico entre estruturas de dados locais, algoritmos clássicos de ordenação/pesquisa e operações persistentes de leitura e escrita.

0.1.2- Objectivos Específicos

- ❖ Modelar entidades do mundo real usando `structs` e gerir coleções indexadas por vetores;
- ❖ Implementar modularidade estrita dividindo o sistema em ficheiros de cabeçalho (`.h`) e código-fonte (`.c`);
- ❖ Desenvolver e justificar a eficácia dos algoritmos *Bubble Sort*, *Selection Sort* e *Insertion Sort*;
- ❖ Resolver as limitações da Pesquisa Binária e criar uma Pesquisa Linear adaptada para detetar múltiplos alunos com o mesmo nome;
- ❖ Criar um sistema estável de I/O em disco que suporte ficheiros de texto estruturados e cópias de segurança binárias compactadas.

0.2- METODOLOGIA DE DESENVOLVIMENTO

O ciclo de desenvolvimento seguiu o modelo incremental e modular:

- ❖ **Modelagem de Dados:** criação e refinamento da `struct Estudante`;
- ❖ **Desenvolvimento Modular da Memória:** criação das funções de cadastro e atualização de vetores em `estudante.c`;
- ❖ **Mecanismo de Persistência:** implementação sequencial dos analisadores de texto (`fscanf/fprintf`) e binários (`fread/fwrite`);
- ❖ **Acoplamento Algorítmico:** inserção dos algoritmos de busca e ordenação parametrizados por ponteiros;
- ❖ **Polimento de Interface e Validação:** padronização visual de consoles para a dimensão exata de **113 caracteres** de largura, tratamento exaustivo de erros de introdução com

funções de limpeza de buffer (`fflush/getchar`) e blindagem contra falhas lógicas no encerramento do sistema.

0.3- DIFICULDADES ENCONTRADAS

Entre os maiores desafios superados na programação em C destacam-se:

- ❖ **Saturação e lixo no Buffer de Entrada (`stdin`):** resolvido através da implementação de uma rotina de limpeza dedicada que limpa caracteres residuais pós-leitura de dados numéricos;
- ❖ **Passagem de Parâmetros e Alteração Real:** compreensão de que a passagem por valor duplicava os dados na pilha e não alterava o contador global de alunos, exigindo a conversão dos parâmetros para referências de ponteiros (`int *total`);
- ❖ **Pesquisa de Nomes Homônimos:** a pesquisa linear padrão parava no primeiro resultado encontrado. Foi necessária uma reengenharia para rastrear todo o vetor, listando todos os alunos com correspondência parcial.

1. CONCEITOS FUNDAMENTAIS UTILIZADOS

1.1- ESTRUTURAS DE DADOS (STRUCTS)

As estruturas em C representam coleções de variáveis heterogêneas sob um único nome e região contígua de memória. Permitem que dados de tipos distintos (como `int` para matrícula, `char[]` para nome e `float` para nota) sejam manipulados de forma unificada.

1.2- ARRAYS (VETORES) E COMPRESSÃO DE MEMÓRIA

Os vetores fornecem armazenamento sequencial e indexado de dados do mesmo tipo em memória RAM. No entanto, a eliminação de dados num vetor estático acarreta o desafio dos vazios operacionais. Para solucionar isto, o **SIGE-UNIKIVI** adota o algoritmo de **Compressão de Memória (Shift à Esquerda)**. Quando um índice i é excluído, todos os elementos subsequentes na posição $i+1$ até $N-1$ são movidos uma célula para trás, garantindo a integridade dos índices sequenciais e permitindo o decréscimo correto do tamanho lógico total da base de dados ($N = N - 1$).

1.3- MODULARIZAÇÃO E PONTEIROS (PASSAGEM POR REFERÊNCIA)

A divisão do código segue o princípio da separação de preocupações (Separation of Concerns).

- ❖ Ficheiros `.h`: Contêm as definições de tipos e protótipos de funções (Contrato).

- ❖ Ficheiros .c: Implementam as rotinas operacionais (Encapsulamento).

Para que as funções modulares consigam alterar os dados declarados no escopo da função `main()`, utiliza-se a **Passagem por Referência**. Em vez de copiar o valor, envia-se o endereço de memória da variável usando o operador `&`. A função recetora utiliza ponteiros (`*`) para ler e escrever diretamente na posição de memória original.

1.4- PERSISTÊNCIA EM FORMATOS TXT E BINÁRIO

O sistema implementa dualidade no armazenamento:

- ❖ **Ficheiro de Texto (.txt):** armazenamento em modo legível por humanos. Útil para auditorias externas e importação manual. Usa formatação controlada para leitura automática linha por linha.
- ❖ **Ficheiro Binário (.bin):** dump direto da memória RAM para o disco. Oferece alta velocidade de leitura/escrita e proteção contra edições acidentais por utilizadores leigos.

2. MODELAGEM DO SISTEMA

2.1- A STRUCT ESTUDANTE

A modelagem lógica da entidade estudante foi fixada com a seguinte assinatura e distribuição de memória em C:

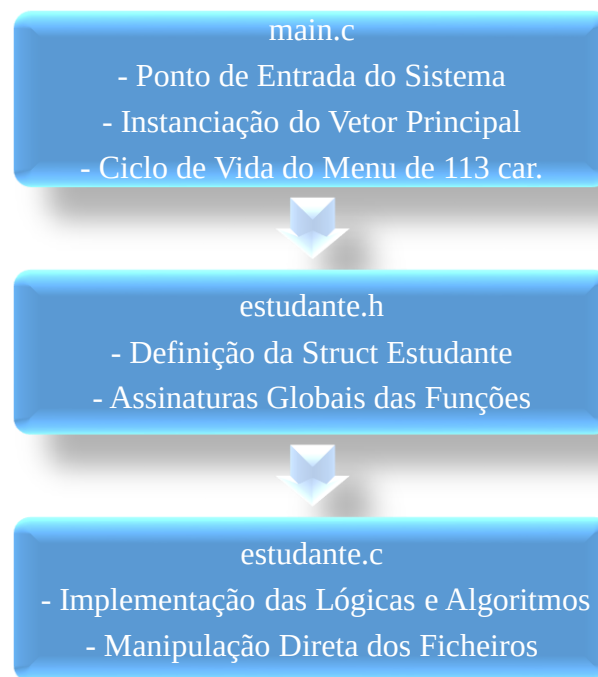
```
typedef struct {  
    int matricula;           // 4 Bytes - Identificador Único Universal  
    char nome[100];         // 100 Bytes - String contendo o nome completo do aluno  
    float nota;             // 4 Bytes - Valor decimal de precisão simples da avaliação  
} Estudante;
```

Total de Memória por Registo: 108 Bytes.

2.2- MAPA DE FLUXO DA ARQUITETURA MODULAR

O projeto encontra-se particionado em 3 divisões fundamentais que se comunicam de forma hierárquica:

Figura nº 01: Mapa De Fluxo Da Arquitetura Modular



Fonte: (Autores, 2026).

3. ANALISE ALGORÍTMICA E IMPLEMENTAÇÃO

3.1- ALGORITMOS DE ORDENAÇÃO

O sistema incorpora os três algoritmos clássicos solicitados pela ementa, permitindo a ordenação da base de dados por múltiplos campos (Matrícula, Nome e Nota):

A. Bubble Sort (Ordenação por Flutuação)

Funciona comparando pares de elementos adjacentes e trocando-os se estiverem na ordem errada. É repetido até que nenhuma troca seja necessária.

- ❖ *Complexidade:* $O(n^2)$ tanto no pior como no melhor caso padrão.
- ❖ *Indicação:* Uso educacional e para vetores quase ordenados.

B. Selection Sort (Ordenação por Seleção)

Divide o vetor em duas partes (ordenada e não ordenada), localiza repetidamente o menor (ou maior) elemento da parte não ordenada e coloca-o no início.

- ❖ *Complexidade:* $\theta(n^2)$ em todos os cenários, devido à necessidade constante de examinar todos os elementos restantes.
- ❖ *Indicação:* Minimiza o número de trocas físicas na memória.

C. Insertion Sort (Ordenação por Inserção)

Semelhante à organização de cartas de baralho nas mãos. Constrói o vetor ordenado final um elemento de cada vez, inserindo cada novo elemento na sua posição correta relativa aos já ordenados.

- ❖ *Complexidade*: Pior Caso $O(n^2)$ | Melhor Caso $O(n)$ quando o vetor já está ordenado.
- ❖ *Indicação*: Altamente eficiente para bases de dados dinâmicas que recebem novos alunos continuamente.

3.2- ALGORITMOS DE PESQUISA

A eficiência na recuperação de dados foi garantida pela aplicação de dois métodos distintos:

A. Pesquisa Linear Avançada (Múltiplos Resultados)

Varre o vetor do índice 0 a N-1 comparando a string procurada com o nome dos estudantes utilizando correspondência de strings (`strstr`). A nossa implementação supera a busca convencional, pois não interrompe a execução ao achar o primeiro estudante; ela rastreia o vetor na íntegra, permitindo exibir todos os estudantes homônimos cadastrados na instituição.

B. Pesquisa Binária (Divisão e Conquista)

Aplicada estritamente sobre o campo chave `matrícula`. O algoritmo exige que o vetor esteja previamente ordenado por matrícula. Divide o espaço de busca a meio a cada iteração:

- ❖ Se a matrícula do meio for igual ao alvo, retorna o índice;
- ❖ Se o alvo for menor, descarta a metade superior;
- ❖ Se for maior, descarta a metade inferior.
- ❖ *Complexidade*: $O(\log n)$, ideal para recuperação instantânea em grandes volumes de dados.

3.3- ALGORITMO DE ELIMINAÇÃO COM CONFIRMAÇÃO

A remoção de um estudante segue uma política restrita de tolerância a falhas operacionais:

- ❖ O utilizador fornece a matrícula do estudante de destino;
- ❖ A pesquisa binária localiza a posição física em memória RAM;
- ❖ Se localizado, o sistema extrai e imprime os dados atuais (Nome e Nota), e emite uma paragem crítica perguntando explicitamente: "Deseja mesmo eliminar? (S/N)";

- ❖ Se confirmado ('s' ou 'S'), o algoritmo executa a compressão de memória deslocando as posições e decrementa o contador global;
- ❖ Retorna um sinal booleano (1), ativando no sistema o aviso de **Alterações Pendentes**, forçando o programa a saber que a memória difere do ficheiro em disco.

4. MANUAL DE UTILIZAÇÃO E RESULTADOS

4.1- INICIALIZAÇÃO E LEITURA DO FICHEIRO

Ao ser inicializado, o **SIGE-UNIKIVI** tenta ler o ficheiro padrão no disco. Se detetado, carrega os dados de forma transparente, preenchendo o vetor e atualizando o contador dinamicamente. Se o ficheiro não for encontrado, o sistema cria o ambiente limpo automaticamente sem interromper a execução.

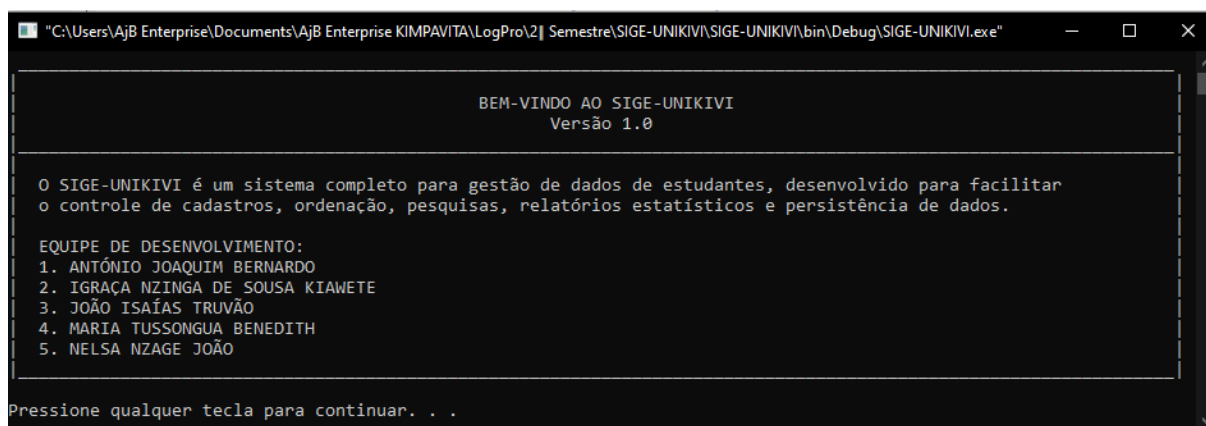
4.2- FLUXO DE SEGURANÇA E ALTERAÇÕES PENDENTES

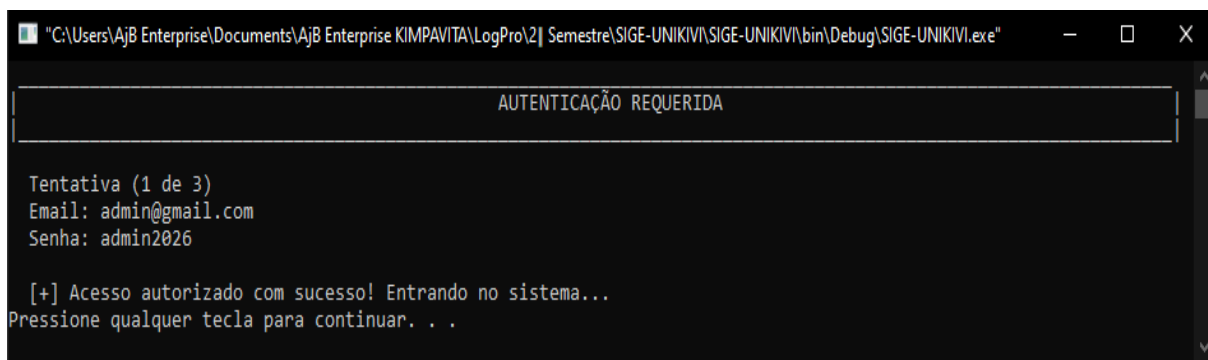
O programa gerencia de forma inteligente o ciclo de salvaguarda de dados. Se o utilizador inserir um aluno ou eliminar um registo e tentar sair imediatamente através da opção 0, o sistema interseta a instrução, bloqueia o encerramento imediato e apresenta um aviso destacado informando que existem modificações na memória RAM que não foram salvas no ficheiro .txt. Dá então a oportunidade de gravar e sair com segurança, eliminando o risco de perda de trabalho por esquecimento.

5.8- APRESENTAÇÃO DOS RESULTADOS

5.8.1- Tela de Login

Figura nº 02: Tela de Login

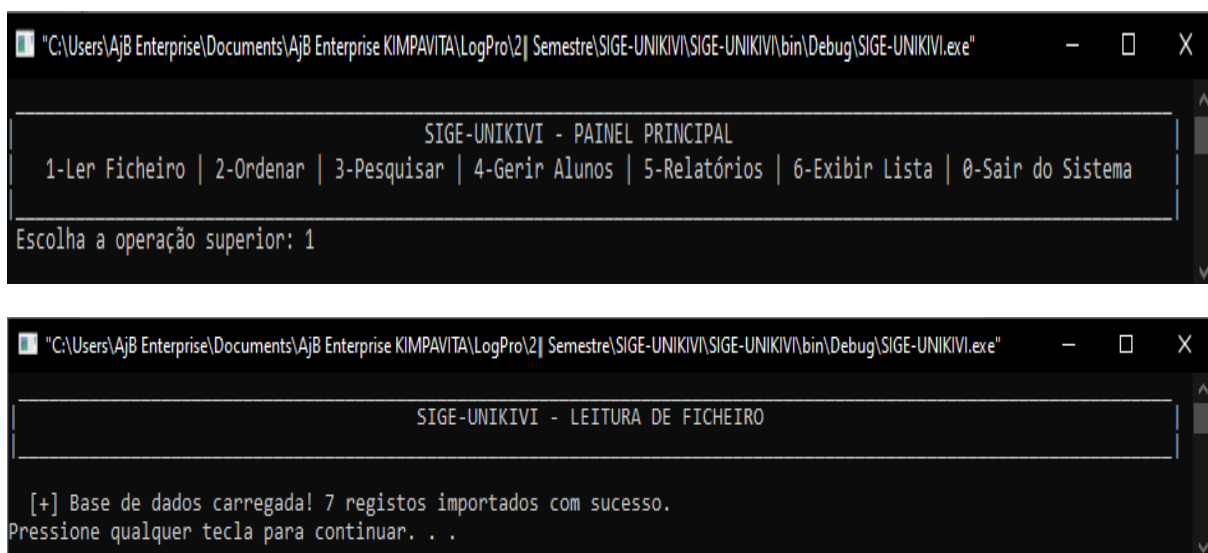




Fonte: (Autores, 2026).

5.8.2- Tela de Menu principal

Figura nº 03: Tela de Menu principal

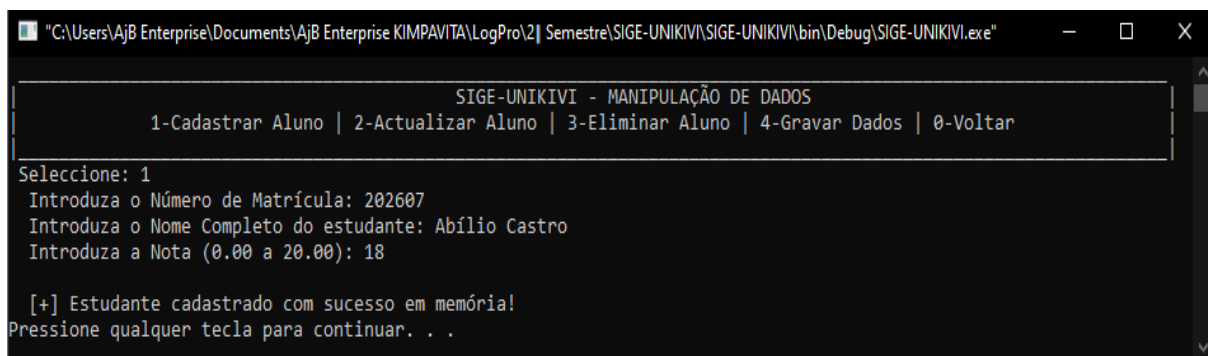


Fonte: (Autores, 2026).

5.8.3- Gerir Alunos

5.8.3.1- Tela de Cadastro

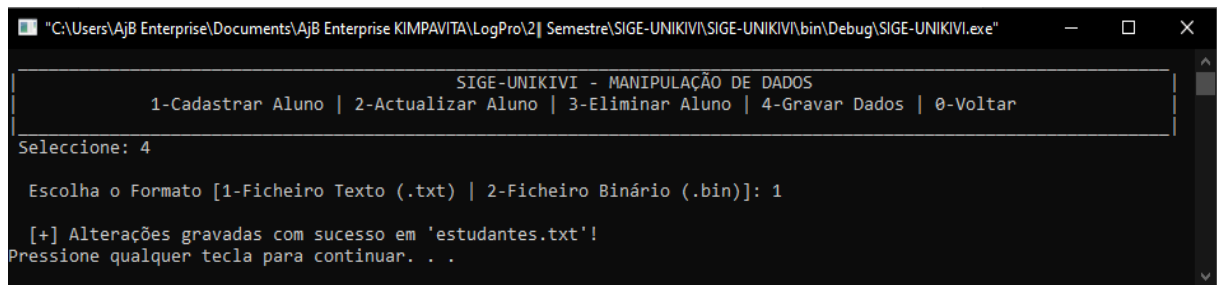
Figura nº 04: Tela de Cadastro



Fonte: (Autores, 2026).

5.8.3.2- Tela de Gravar dados

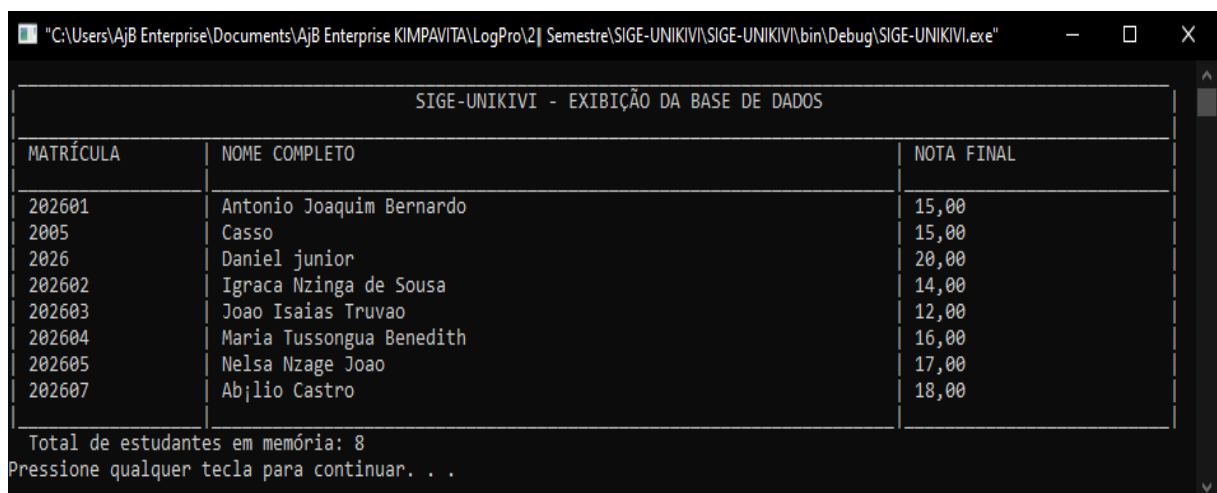
Figura nº 05: Tela de Gravar dados



Fonte: (Autores, 2026).

5.8.4- Tela de Listagem de alunos

Figura nº 06: Tela de Listagem de alunos



MATRÍCULA	NOME COMPLETO	NOTA FINAL
202601	Antonio Joaquim Bernardo	15,00
2005	Casso	15,00
2026	Daniel junior	20,00
202602	Igraca Nzinga de Sousa	14,00
202603	Joao Isaias Truvao	12,00
202604	Maria Tussongua Benedith	16,00
202605	Nelsa Nzage Joao	17,00
202607	Abílio Castro	18,00

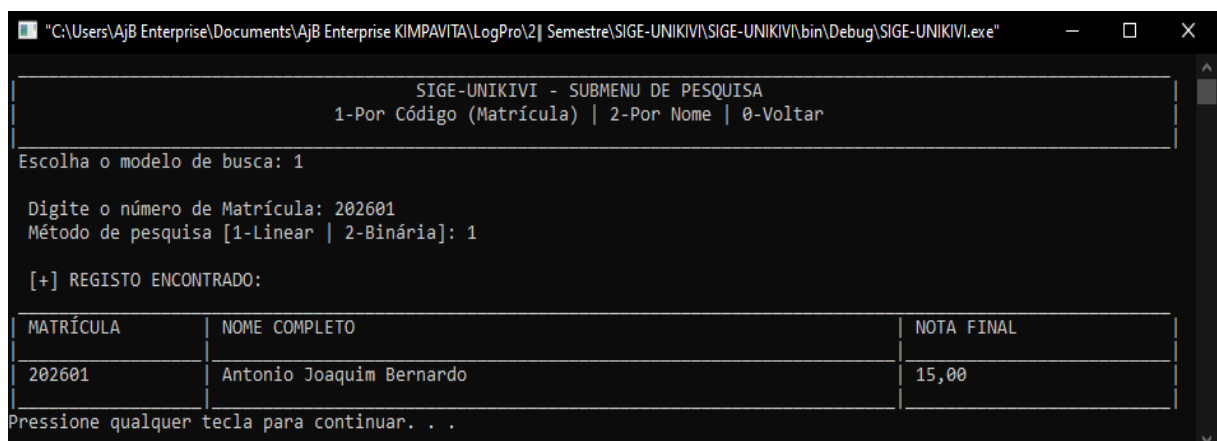
Total de estudantes em memória: 8

Pressione qualquer tecla para continuar. . .

Fonte: (Autores, 2026).

5.8.5- Tela de Pesquisa por Matrícula

Figura nº 07: Tela de Pesquisa por matrícula



SIGE-UNIKIVI - SUBMENU DE PESQUISA

1-Por Código (Matrícula) | 2-Por Nome | 0-Voltar

Escolha o modelo de busca: 1

Digite o número de Matrícula: 202601

Método de pesquisa [1-Linear | 2-Binária]: 1

[+] REGISTO ENCONTRADO:

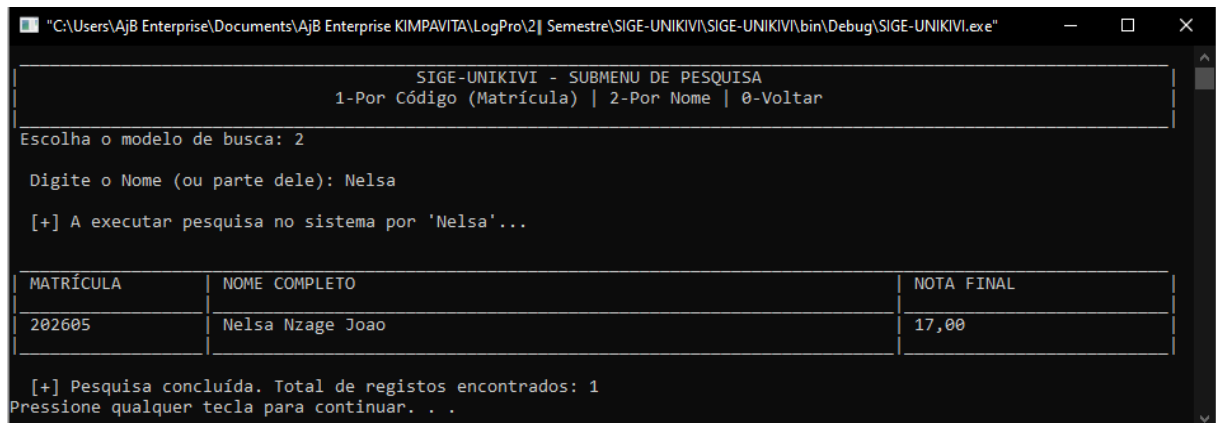
MATRÍCULA	NOME COMPLETO	NOTA FINAL
202601	Antonio Joaquim Bernardo	15,00

Pressione qualquer tecla para continuar. . .

Fonte: (Autores, 2026).

5.8.6- Tela de Pesquisa por nome

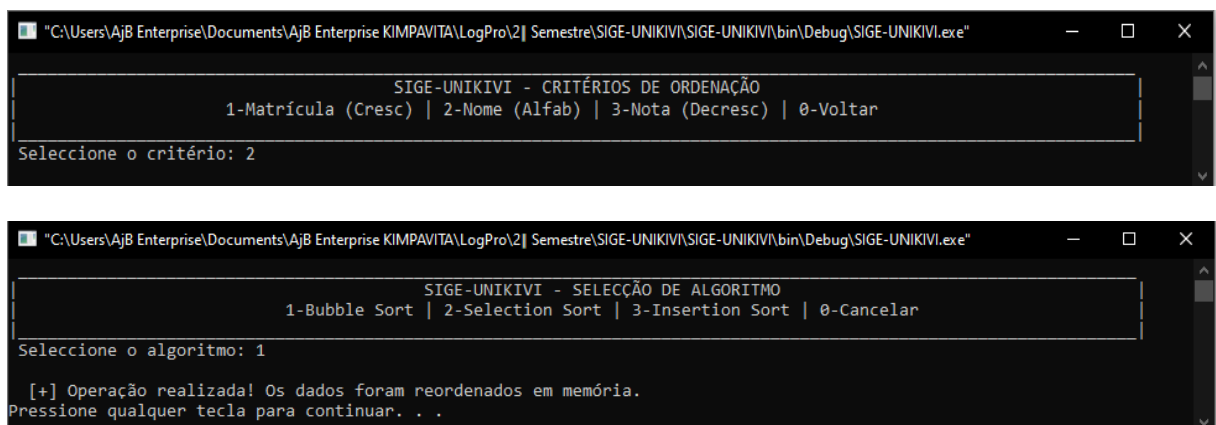
Figura nº 08: Tela de Pesquisa por nome



Fonte: (Autores, 2026).

5.8.7- Tela de Ordenação

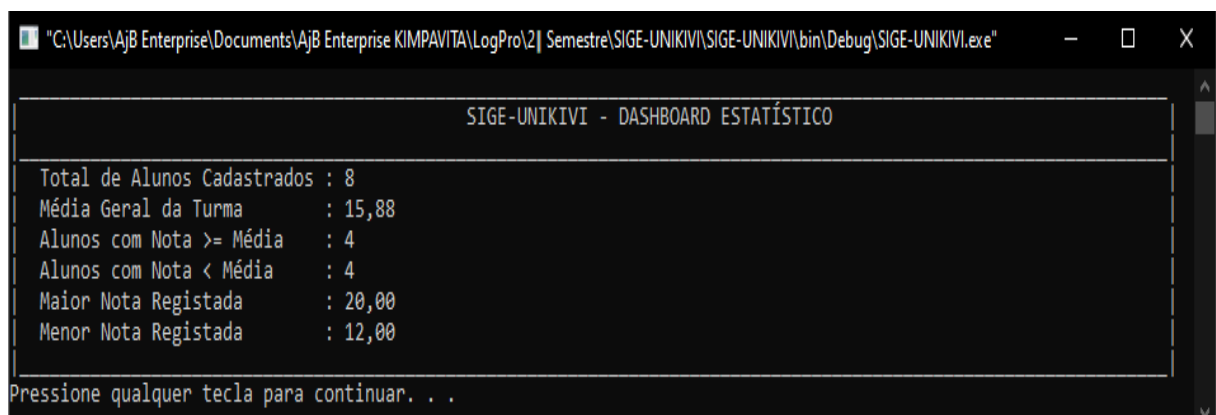
Figura nº 9: Tela de ordenação



Fonte: (Autores, 2026).

5.8.8- Tela de Relatórios

Figura nº 10: Tela de Relatórios



Fonte: (Autores, 2026).

CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema **SIGE-UNIKIVI** proporcionou uma consolidação prática profunda acerca do funcionamento da gestão de memória e persistência em sistemas computacionais de baixo nível utilizando a linguagem C. A implementação modular e o encapsulamento em ficheiros fonte separados provaram ser fundamentais para viabilizar a manutenção contínua do software sem gerar efeitos secundários indesejados.

Adicionalmente, a análise comparativa entre os algoritmos de ordenação evidenciou que a escolha de uma rotina de ordenação ou pesquisa não deve ser aleatória, mas baseada nas restrições de tempo e no comportamento esperado da base de dados. A introdução de rotinas avançadas, como a compressão física de vetores pós-exclusão e as caixas de diálogo de confirmação explícita, transformaram o projeto académico numa aplicação segura e alinhada com as melhores práticas de Engenharia de Software.

REFERÊNCIAS BIBLIOGRÁFICAS

KERNIGHAN, Brian W.; RITCHIE, Dennis M. *The C Programming Language*. 2ª Edição. Englewood Cliffs, NJ: Prentice Hall, 1988. (Referência fundamental da linguagem padrão ANSI C).

LAFORE, Robert. *Estruturas de Dados e Algoritmos em C++*. Rio de Janeiro: Ciência Moderna, 2004. (Fundamentos de vetores, compressão e algoritmos de ordenação).

MANZANO, José Augusto N. G. *Algoritmos: Lógica para Desenvolvimento de Programação de Computadores*. 22ª Edição. São Paulo: Érica, 2010.

EBERSPÄCHER, Henri Frederico. *Lógica de Programação*. São Paulo: Pearson Prentice Hall, 2005.